Single Day Analysis (Ravel CSTs)

```
# =======
# Imports
# =======

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from scipy.stats import gmean
from scipy.spatial.distance import pdist, squareform

from sklearn.decomposition import PCA

from skbio.stats.distance import DistanceMatrix, permanova, permdisp
```

```
# ==============
# Load and Merge
# ==============

samples = pd.read_parquet("hugerth_sample_profiles.parquet")

meta = pd.read_csv("meta_data.csv", encoding="utf-8-sig")
meta_sub = meta[["Run", "Subject", "Visit_No."]].rename(columns={"Run": "sample_id"})
samples = samples.merge(meta_sub, on="sample_id", how="left")

cst = pd.read_pickle("ravel_csts.df").reset_index().rename(columns={"Sample": "sample_id"})
samples = samples.merge(cst[["sample_id", "CST"]], on="sample_id", how="left")

samples = samples.dropna(subset=["CST"]).copy()
samples["CST"] = samples["CST"].astype(int)

aa_columns = ["Ala","Arg","Asn","Asp","Cys","Gln","Glu","Gly","His","Ile",
    "Leu","Lys","Met","Phe","Pro","Ser","Thr","Trp","Tyr","Val"]
pct_columns = [aa + "_pct" for aa in aa_columns]
```

```
# =================
# Selecting Top Day
# =================

day_counts = samples["Visit_No."].value_counts()
print(day_counts.head())

top_day = day_counts.index[0]
day = samples[samples["Visit_No."] == top_day].copy()
print(f"selected {top_day}: {len(day)} samples")
print(day["CST"].value_counts().sort_index())
```

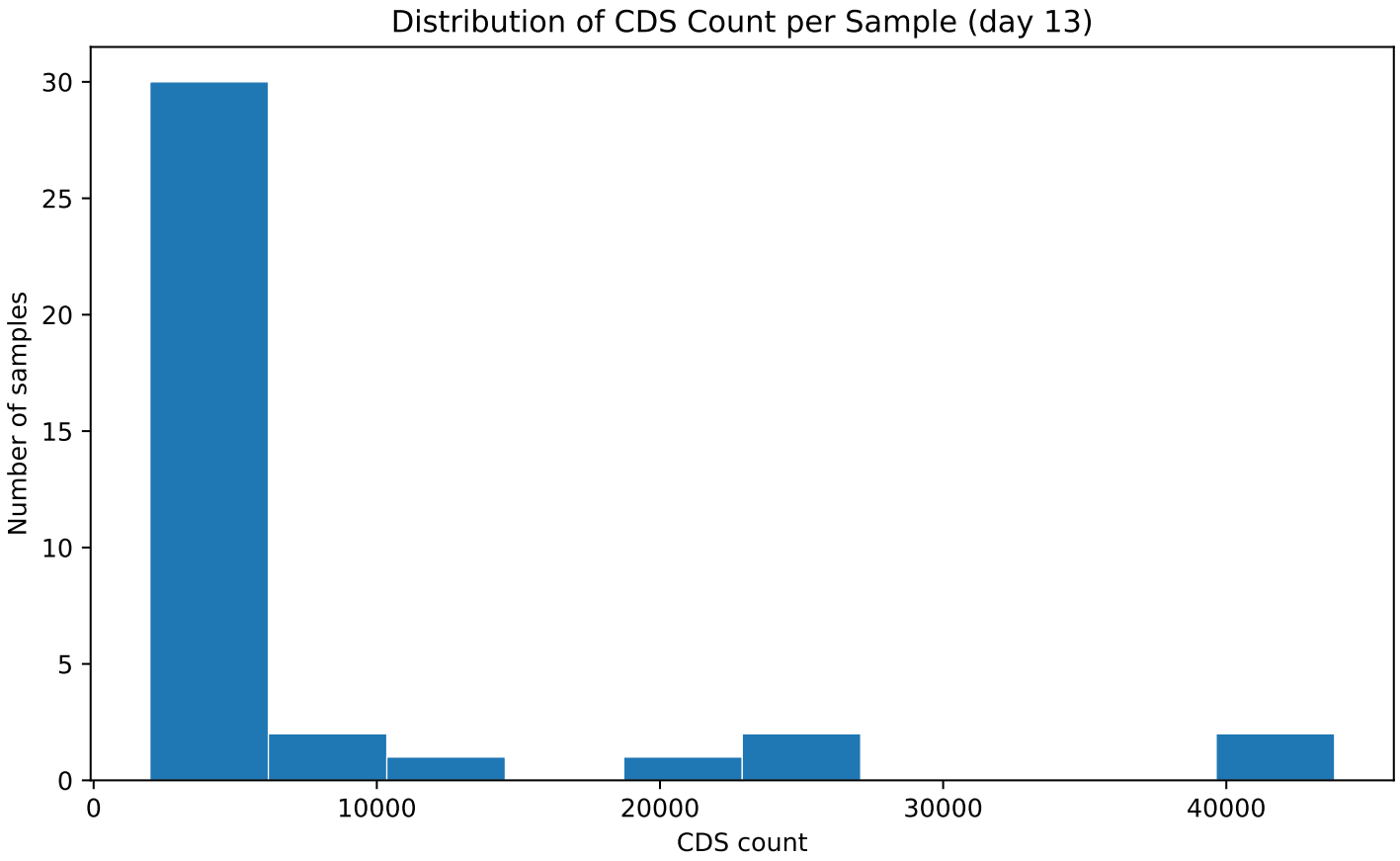
```
Visit_No.
day13    38
day26    38
day28    38
day23    37
day24    37
Name: count, dtype: int64
selected day13: 38 samples
CST
1    21
3     6
4    11
Name: count, dtype: int64
```

```
X_day = day[pct_columns].values
P_day = np.where(X_day == 0, X_day[X_day > 0].min()/2, X_day)
X_clr_day = np.log(P_day/gmean(P_day, axis=1, keepdims=True))

cst_levels = sorted(day["CST"].unique())
cmap_cst = plt.get_cmap("tab10")
cst_colors = {c: cmap_cst(i) for i, c in enumerate(cst_levels)}
```

```
# ====================
# CDS Count Histogram
# ====================

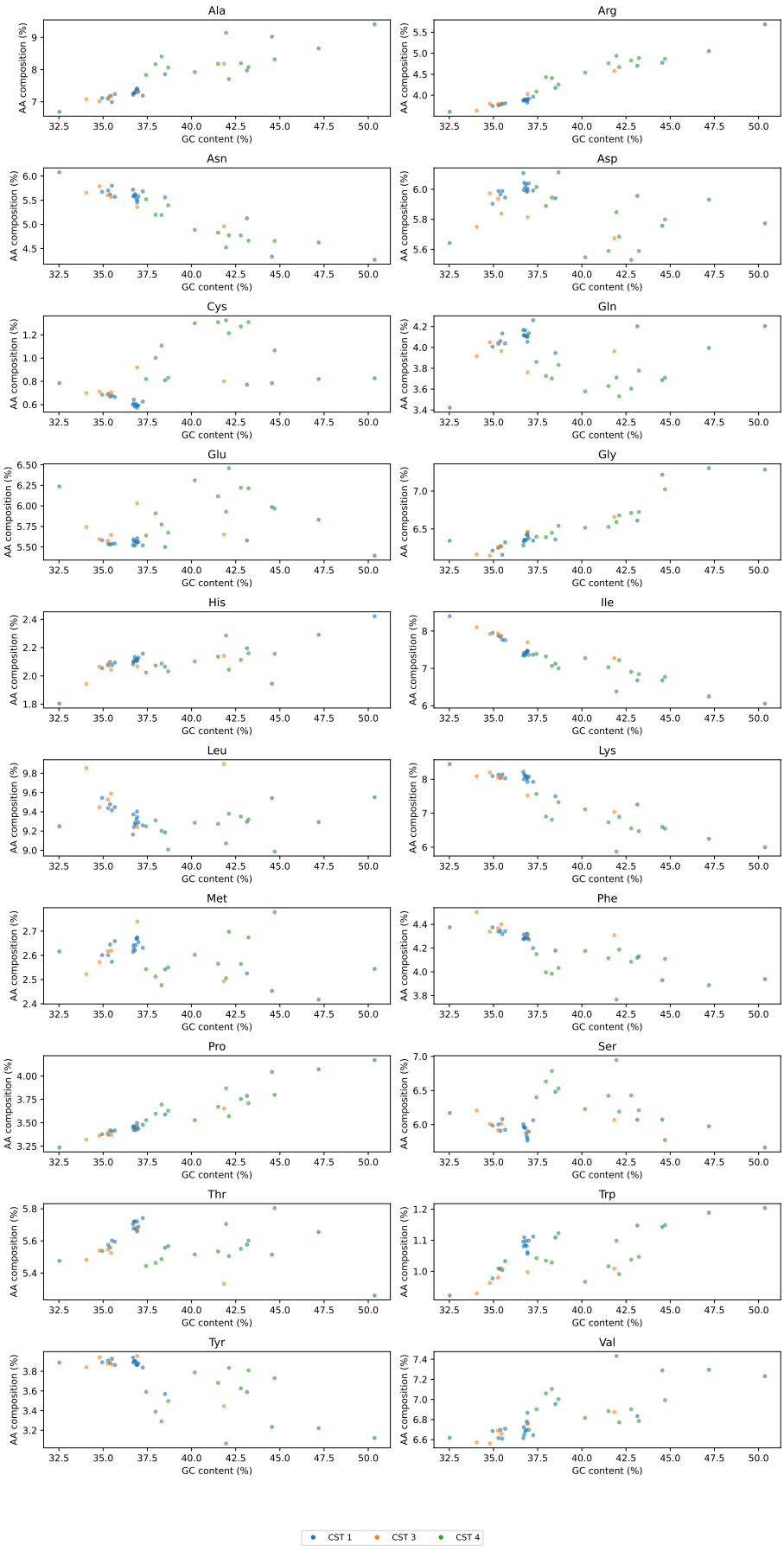
fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(day["n_cds"], bins=10, edgecolor="white", linewidth=0.5)
ax.set_title("Distribution of CDS Count per Sample (day 13)")
ax.set_xlabel("CDS count")
ax.set_ylabel("Number of samples")
plt.tight_layout()
plt.show()
```




```
# =====================
# GC vs. AA composition
# =====================

fig, axes = plt.subplots(10, 2, figsize=(12, 24))
axes = axes.flatten()
for ax, aa, pct_col in zip(axes, aa_columns, pct_columns):
    for c in cst_levels:
        m = day["CST"] == c
        ax.scatter(day.loc[m, "GC_content"], day.loc[m, pct_col],
                   color=cst_colors[c], alpha=0.6, s=20, linewidths=0)
    ax.set_title(aa)
    ax.set_xlabel("GC content (%)")
    ax.set_ylabel("AA composition (%)")

handles = [plt.Line2D([0],[0], marker="o", color="w",
           markerfacecolor=cst_colors[c], markersize=6, label=f"CST {c}")
           for c in cst_levels]
fig.legend(handles=handles, loc="lower center", ncol=len(cst_levels), fontsize=9,
           bbox_to_anchor=(0.5, -0.01))
fig.suptitle("GC Content vs Amino Acid Composition by CST (day 13)", fontsize=16, y=1.0)
plt.tight_layout(rect=[0, 0.03, 1, 0.98])
plt.show()
```

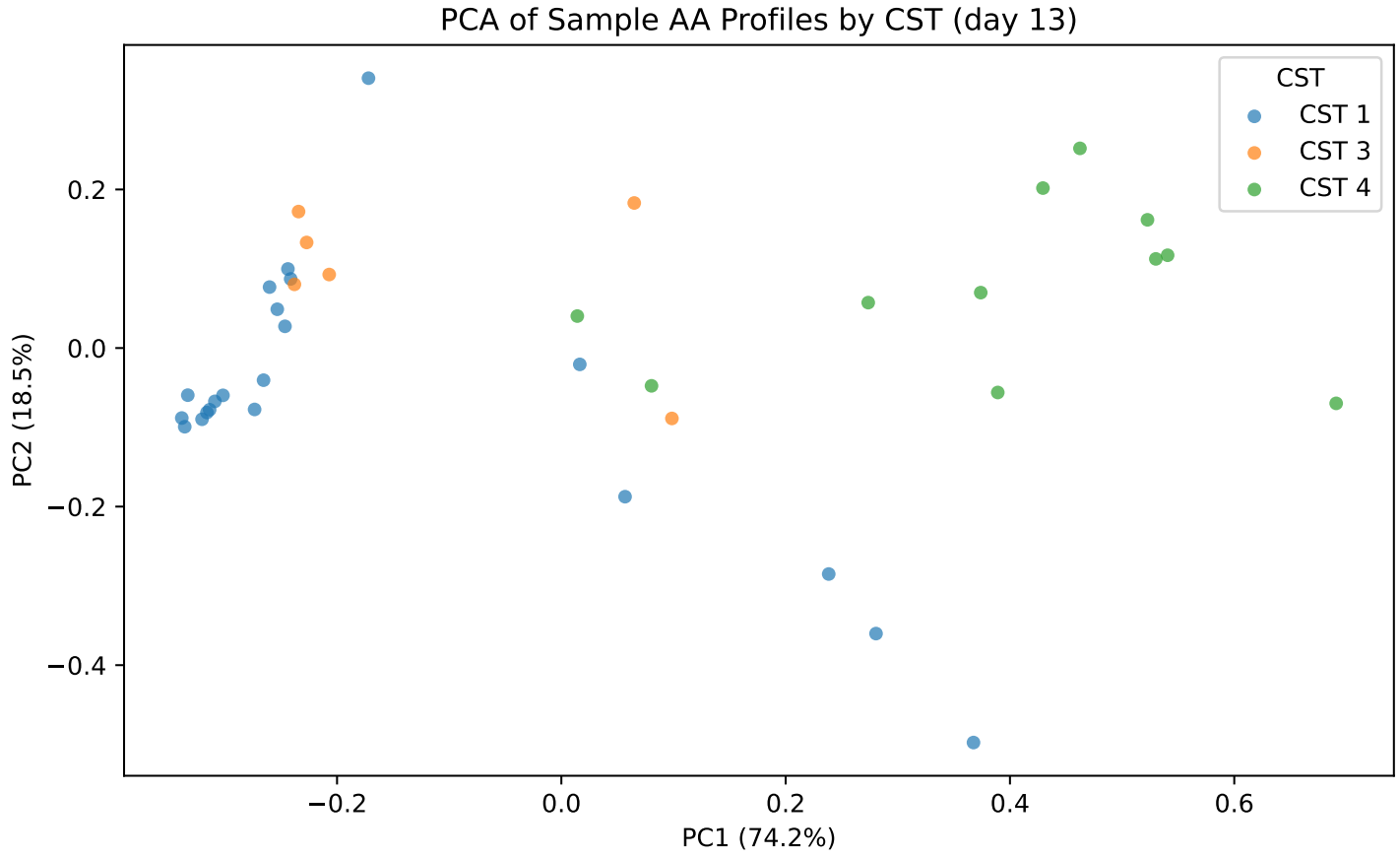

GC Content vs Amino Acid Composition by CST (day 13)
Ala
Arg
Asn
Asp
Cys
Gln
Glu
Gly
His
Ile
Leu
Lys
Met
Phe
Pro
Ser
Thr
Trp
Tyr
Val
AA composition (%)
GC content (%)
CST 1
CST 3
CST 4

```
# ======================
# PCA color coded by CST
# ======================

pca_day = PCA()
X_pca_day = pca_day.fit_transform(X_clr_day)

evr_day = pca_day.explained_variance_ratio_

fig, ax = plt.subplots(figsize=(8, 5))
for c in cst_levels:
    m = (day["CST"] == c).values
    ax.scatter(X_pca_day[m, 0], X_pca_day[m, 1], color=cst_colors[c],
               alpha=0.7, s=30, linewidths=0, label=f"CST {c}")
ax.set_title("PCA of Sample AA Profiles by CST (day 13)")
ax.set_xlabel(f"PC1 ({evr_day[0]:.1%})")
ax.set_ylabel(f"PC2 ({evr_day[1]:.1%})")
ax.legend(title="CST")
plt.tight_layout()
plt.show()
```




```
# ==============
# PCA Scree Plot
# ==============

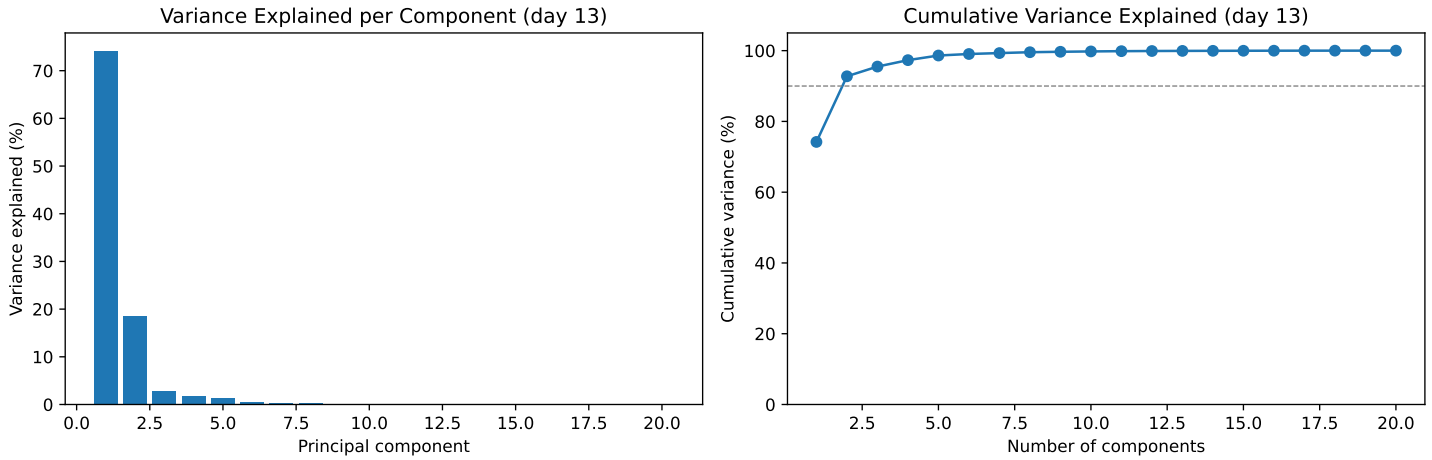
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].bar(range(1, len(evr_day)+1), 100*evr_day)
axes[0].set_title("Variance Explained per Component (day 13)")
```

```
axes[0].set_xlabel("Principal component")
axes[0].set_ylabel("Variance explained (%)")

axes[1].plot(range(1, len(evr_day)+1), 100*evr_day.cumsum(), marker="o")
axes[1].axhline(90, linestyle="--", linewidth=0.8, color="gray")
axes[1].set_title("Cumulative Variance Explained (day 13)")
axes[1].set_xlabel("Number of components")
axes[1].set_ylabel("Cumulative variance (%)")
axes[1].set_ylim(0, 105)

plt.tight_layout()
plt.show()
```

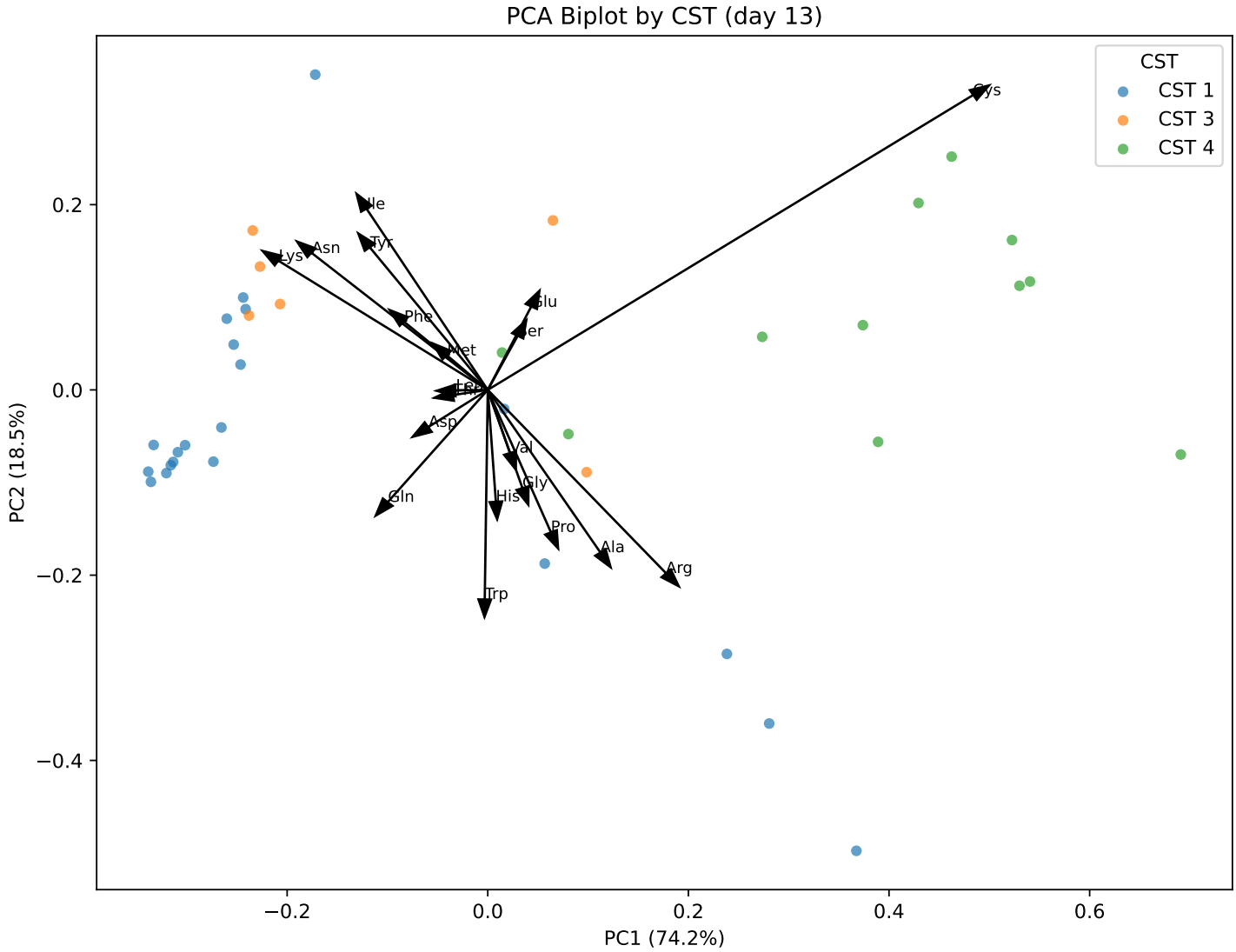



```
# ==========
# PCA Biplot
# ==========

load_day = pca_day.components_[:2].T

span = max(abs(X_pca_day[:,0]).max(), abs(X_pca_day[:,1]).max())
scale = 0.7*span/abs(load_day).max()
hw = 0.02*span

fig, ax = plt.subplots(figsize=(9, 7))
for c in cst_levels:
    m = (day["CST"] == c).values
    ax.scatter(X_pca_day[m,0], X_pca_day[m,1], color=cst_colors[c],
               alpha=0.7, s=30, linewidths=0, label=f"CST {c}")
for i, aa in enumerate(aa_columns):
    ax.arrow(0, 0, load_day[i,0]*scale, load_day[i,1]*scale,
             linewidth=0.7, color="black", head_width=hw)
    ax.annotate(aa, (load_day[i,0]*scale, load_day[i,1]*scale), fontsize=8)
ax.set_title("PCA Biplot by CST (day 13)")
ax.set_xlabel(f"PC1 ({evr_day[0]:.1%})")
ax.set_ylabel(f"PC2 ({evr_day[1]:.1%})")
ax.legend(title="CST")
plt.tight_layout()
plt.show()
```




```
# =========
# PERMANOVA
# =========

D_day = squareform(pdist(X_clr_day, metric="euclidean"))
dm_day = DistanceMatrix(D_day, ids=day["sample_id"].astype(str).values)

res = permanova(dm_day, day["CST"].astype(str).values, permutations=9999)
print(res)
print()
res_disp = permdisp(dm_day, day["CST"].astype(str).values, permutations=9999)
print(res_disp)
```

```
method name                PERMANOVA
test statistic name         pseudo-F
sample size                       38
number of groups                   3
test statistic             17.458292
p-value                       0.0001
number of permutations          9999
Name: PERMANOVA results, dtype: object
```

```
method name                PERMDISP
test statistic name         F-value
sample size                      38
number of groups                  3
test statistic             0.301981
p-value                      0.6946
number of permutations         9999
Name: PERMDISP results, dtype: object
```